



PROJETO EDUCAÇÃO
DO FUTURO

Banco de Dados - Parte 3



Sumário

- JOINS
- Modelo Entidade Relacionamento (MER)
- Cardinalidades
- Chave Primária
- Chave Estrangeira



Relembrando

Na última aula vimos como podemos criar nossas tabelas, bases de dados e inserir informações dentro do nosso banco.

Agora iremos continuar falando de mais alguns comandos do sql que são úteis no dia a dia, além de conhecermos o conceito de chave primária e estrangeira.

Case... When

O comando Case When funciona de forma igual a um “IF” no python, onde vamos passando por várias condições e quando uma delas é verdadeira iremos realizar um determinado trecho de código.

Ela é separada da seguinte maneira:

CASE: Inicia nossa estrutura condicional.

WHEN: Essa palavra permite a nós criamos uma condição logo após seu uso.

THEN: Caso a condição dentro do **when** for verdadeiro irá realizar o que vier após o **then**.

Case When... ELSE

Caso a nossa condição dentro do when não seja atendida iremos cair dentro do **ELSE**.

Ademais o comando case precisa obrigatoriamente se encerrado com o comando **END**.

```
SELECT OrderID, Quantity,  
CASE  
    WHEN Quantity > 30 THEN "The quantity is greater than 30"  
    WHEN Quantity = 30 THEN "The quantity is 30"  
    ELSE "The quantity is under 30"  
END  
FROM OrderDetails;
```

Atividade

1. Crie uma tabela chamada aluno com os campos

Nome	Nota
VARCHAR(50)	INT

1. Agora popule essa tabela com 10 registros
1. Realize um CASE WHEN no campo de nota e se o aluno tiver tirado uma nota maior que 7 exiba aprovado caso contrário reprovado

IF()

O if é basicamente o comando que já conhecemos do python, ele pode servir como uma alternativa ao CASE WHEN.

```
SELECT  
    OrderID,  
    Quantity,  
    IF(Quantity>10, "MORE", "LESS")  
FROM OrderDetails;
```

Titanic

1. Entre nesse [link](#)
1. Pegue o script que está dentro do arquivo e rode no MySQL.

Titanic - Explicação

- A base de dados que iremos trabalhar é uma tabela resumo referente ao titanic, onde contém informações sobre as pessoas que estavam dentro do navio quando o acidente aconteceu.
- As colunas da tabela tem o seguinte significado:

Titanic - Explicação

survival - Sobreviveu? (0 = Não; 1 = Sim)

Pclass - Classe do passageiro (1 = 1st; 2 = 2nd; 3 = 3rd)

name - Nome do passageiro

sex - Sexo

age - Idade

sibsp - Número de irmãos /Esposas(os) a bordo

parch - Número de pais/Filhos a Bordo

Titanic - Explicação

ticket – Número do ticket para embarcar no navio

fare – Custo da passagem

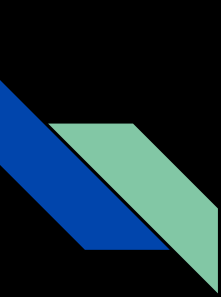
cabin – Cabine do passageiro

embarked – Local do porto onde o passageiro embarcou
(C = Cherbourg; Q = Queenstown; S = Southampton)



Titanic - Atividades

1. Filtre os passageiros com a idade (age) acima de 20.
1. Filtre todos os passageiros do sexo Masculino que morreram.
1. Faça um CASE WHEN para a coluna de **survived** e coloque se a pessoa está “vivo” ou “morto”.
1. Faça um CASE WHEN na coluna de Pclass e coloque a classe do passageiro correspondente ao seu número.
1. Faça um CASE WHEN na coluna de Embarked e coloque o nome do porto que a pessoa saiu.



Titanic - Atividades

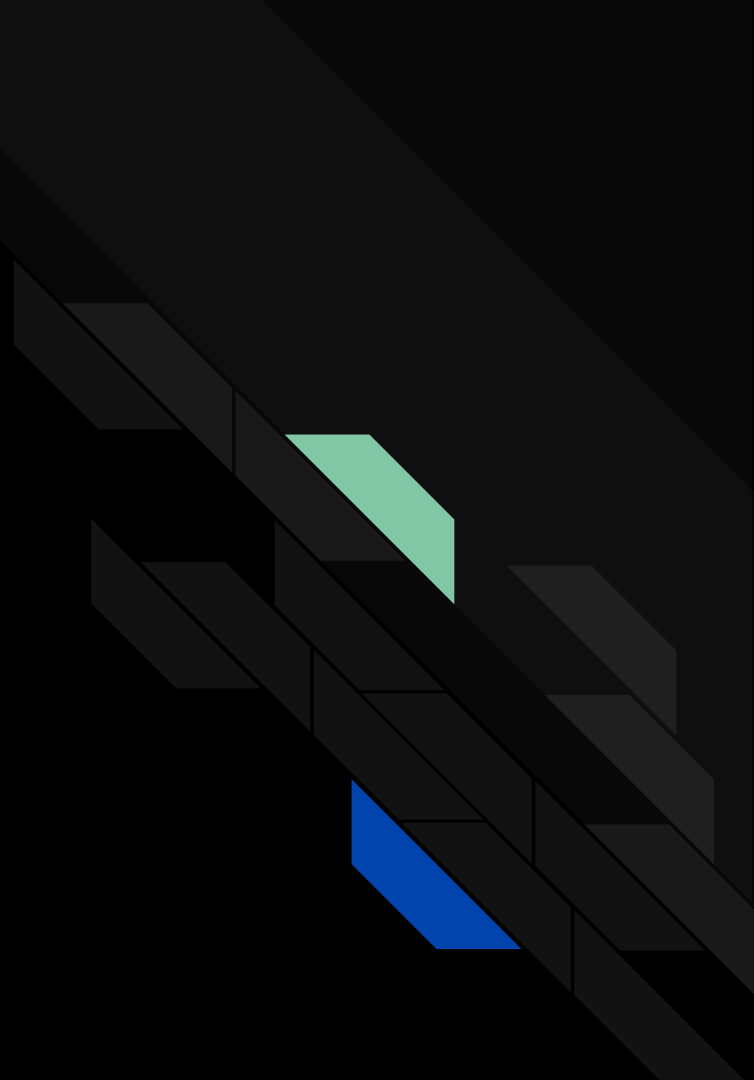
1. Filtre todos os passageiros do sexo feminino que
1. Filtre todos os passageiros do sexo masculino que estão na primeira classe.
1. Qual a média de idade dos passageiros?
1. Qual a média de idade dos passageiros sobreviventes do sexo masculino?
1. Qual a média de idade dos passageiros não sobreviventes do sexo feminino?

Recapitulando...

Na última aula utilizamos os comandos mais comuns do SQL para criar uma tabela e como podemos inserir, modificar e até mesmo deletar elementos da nossa tabela.

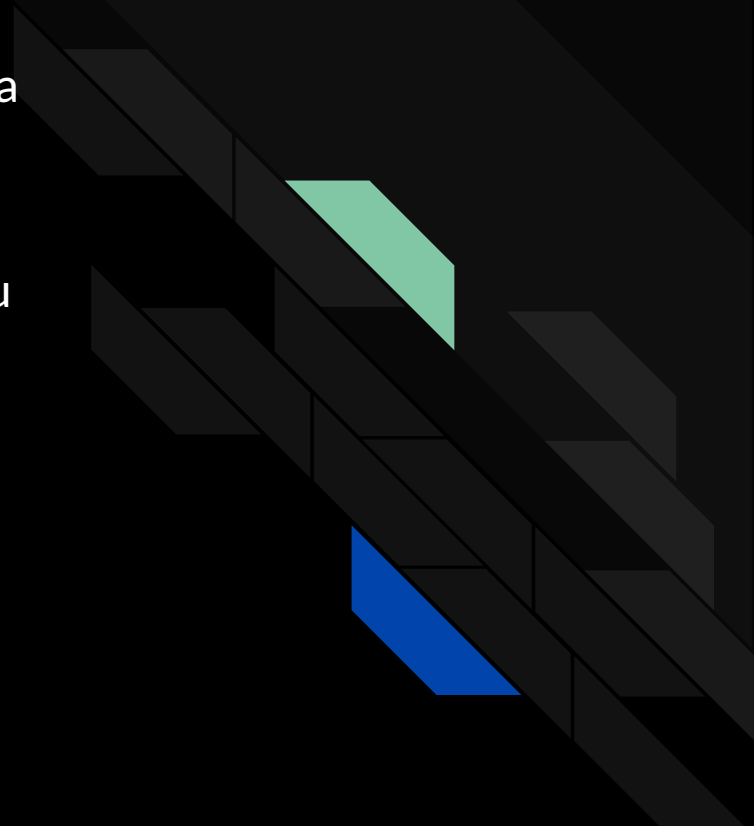
Agora iremos continuar falando da chave primária e conhecer um novo conceito que tem a mesma importância que é a chave estrangeira.

Hoje iremos conhecer a chave estrangeira e sua importância para o modelo **MER** (Modelo Entidade Relacionamento)



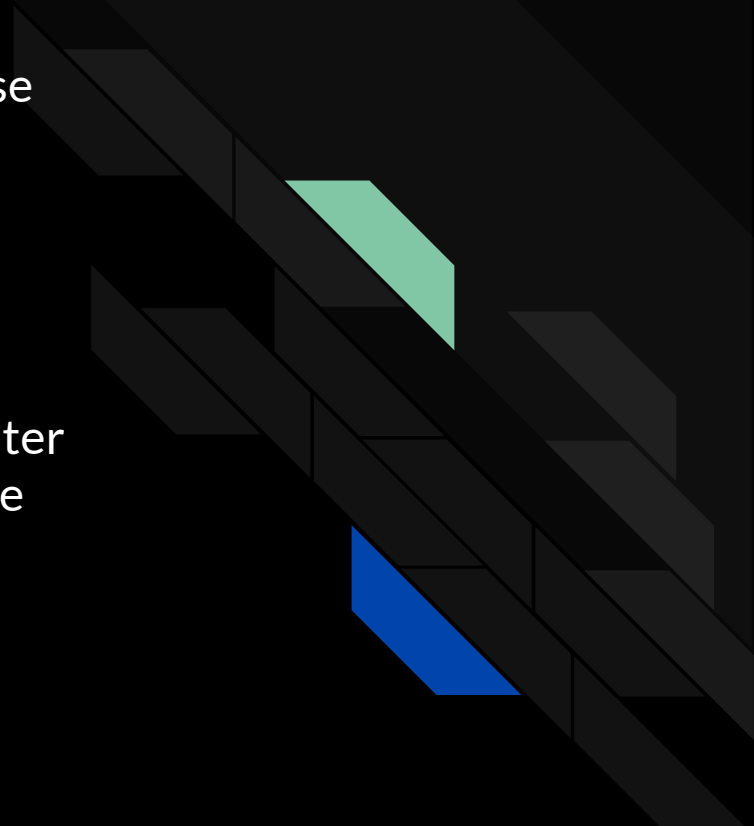
Modelo Entidade Relacionamento (MER)

- O Modelo Entidade Relacionamento serve para representarmos nossas entidades(tabelas) e como elas se relacionam umas com as outras.
- Em um sistema de compra e venda de carros eu posso representar o meu cliente(entidade) por meio de uma tabela 'Cliente' e o carro como uma outra entidade(tabela) 'Carro'
- Cada um deles terá suas características, mas existe uma coisa importante entre eles... Um cliente poderá comprar mais de um carro!



Modelo Entidade Relacionamento (MER)

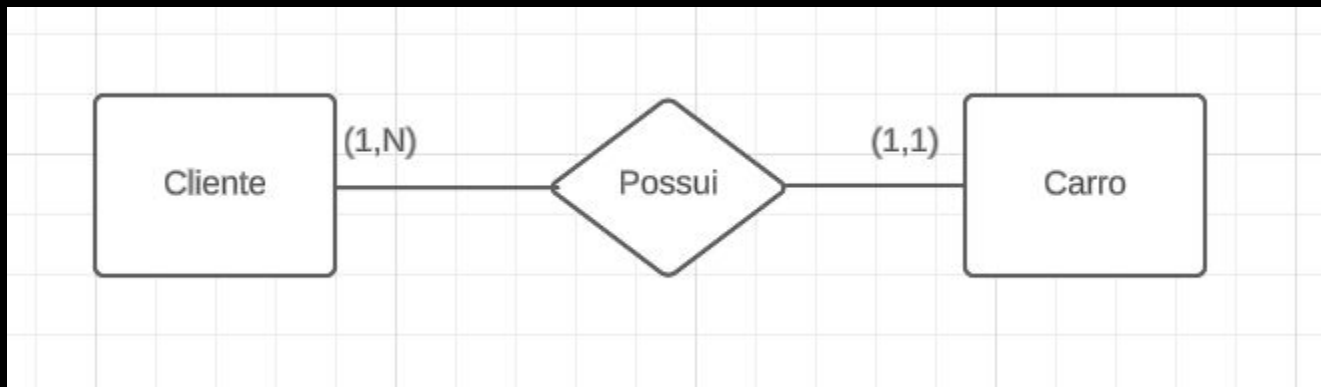
- Esse fato de duas tabelas diferentes poderem se relacionar é chamado de relacionamento.
- Pense no seguinte: Você é um cliente e deseja adquirir um carro, mas... quantos carros você pode ter?
- A resposta para isso é 'N', pois um cliente pode ter 'N' carros, pois não existe nenhuma restrição de quantos carros ele pode ter.
- Esse fato do cliente poder comprar 'N' carros é chamado de cardinalidade



Cardinalidades 1-N

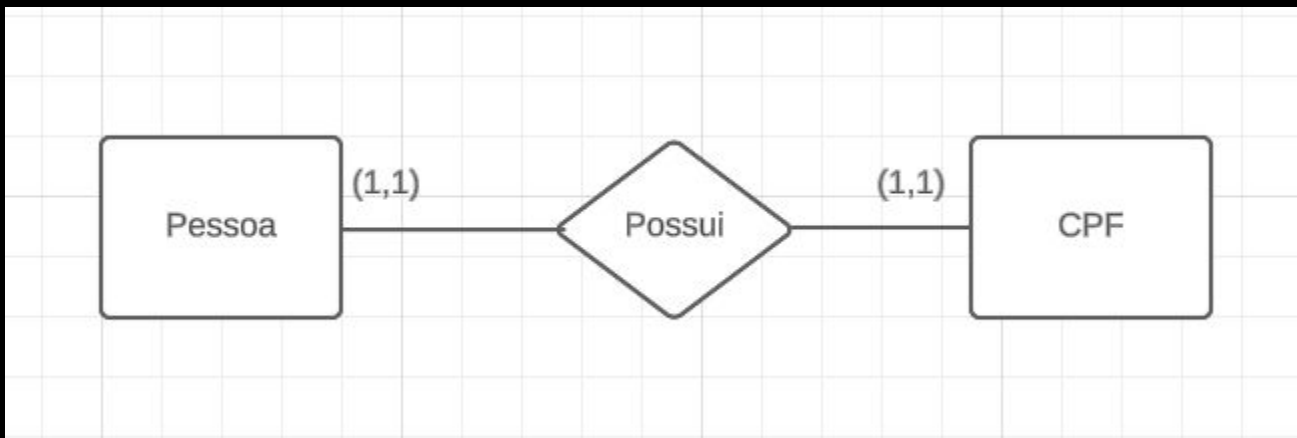
Cardinalidade é o conceito utilizado para representar qual o grau de relacionamentos de uma entidade para com outra

Se formos representar isso no formato de diagrama ficaria dessa maneira:



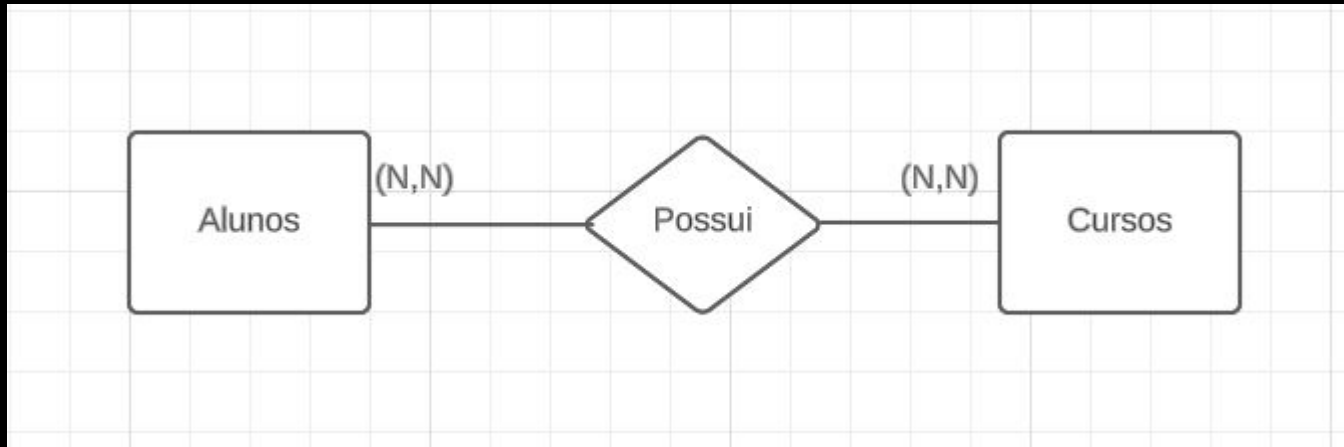
Cardinalidades 1-1

Existem diversos tipos de cardinalidade que podemos utilizar para representar o relacionamento de nossas tabelas uma que já vimos foi a cardinalidade 1-N, porém existem outros tipos de cardinalidade como por exemplo a cardinalidade 1 para 1



Cardinalidade N-N

Existe também a cardinalidade de N para N, ou seja, muito para muitos que podemos representar da seguinte maneira:



Chave Primária

Como já vimos anteriormente, a chave primária funciona como um localizador universal que permite achar alguém ou alguma coisa independente do tamanho da minha tabela.

Geralmente nomeamos essa coluna que é a nossa chave primária como 'ID'. Uma tabela só pode possuir uma chave primária.

Lembrando que chaves primárias não podem se repetir.



Atividade

1. Crie uma base de dados com o **create database** chamada de 'concessionária'.
1. Utilize o comando '**USE** concessionária' para entrar dentro da sua base de dados.
1. Crie uma tabela chamada Cliente que terá as seguintes colunas:

ID_Cliente	Nome	Bairro	Cidade
INT PK AI	VARCHAR(50)	VARCHAR(50)	VARCHAR(50)

- Agora popule essa tabela com 10 registros

Chave Estrangeira

Uma chave estrangeira nada mais é que uma coluna (chave) de uma tabela que se conecta a uma outra tabela (mas especificamente em sua chave primária)

Uma chave estrangeira é uma representação de um relacionamento entre duas tabelas

Tal qual como a chave primária a estrangeira possui algumas regras:

- Pode ter mais de uma chave estrangeira por tabela
- Podem conter valores duplicados
- Podem existir valores nulos

Como adicionar uma chave estrangeira em uma tabela

Podemos adicionar uma chave estrangeira da seguinte forma:

```
CREATE TABLE TabelaExemplo(  
  Coluna1 int  
  Coluna2 PK AI  
  Coluna3 Varchhar(50)  
  ...  
  FOREIGN KEY (fk_id_cliente) REFERENCES Clientes(ID_cliente)  
)
```

Atividade

- Crie uma tabela chamada Carros e coloque nela as seguinte colunas:

```
CREATE TABLE Carros (  
    ID_carro INT PRIMARY KEY AUTO_INCREMENT,  
    FK_ID_Cliente INT,  
    Nome_Carro VARCHAR(50),  
    Marca VARCHAR(50),  
    FOREIGN KEY (FK_ID_Cliente) REFERENCES Clientes(ID_Cliente)  
);
```


Atividade

- Agora preencha a tabela de carros com 15 registros
- **OBS:** Na 'FK_ID_Cliente' é necessário preencher ela com valores que existem na tabela 'Cliente' na coluna 'ID_Cliente', pois iremos associar a chave primária dessa pessoa com a Chave Estrangeira.

JOINS

Os joins são maneiras que temos de juntar nossas tabelas que possuem chaves estrangeiras e chaves primárias que correspondem com determinada consulta que desejamos realizar.

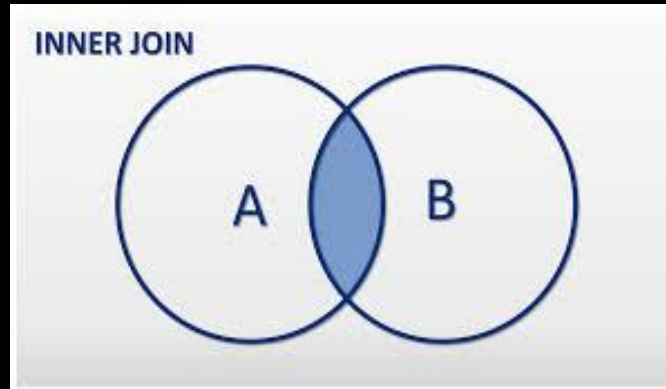
Tome como exemplo que você deseja selecionar todos os clientes da sua tabela 'cliente' que já realizaram uma compra da sua tabela produtos. Como poderíamos fazer para juntar essas duas informações que ficam em tabelas diferentes?

A resposta para isso são os JOINS (Junção) que permite que possamos juntar essas informações durante a nossa consulta e gerar uma nova tabela com informação das duas tabelas selecionadas.

INNER JOIN

Provavelmente o JOIN mais conhecido do SQL, pois ele permite juntarmos duas ou mais tabelas para podermos realizar nossa consulta

Ele irá pegar uma coluna que existe na tabela A e comparar com outra coluna na tabela B (ou vice-versa) e irá nos retornar as linhas que forem compatíveis (ou seja, um registro de uma coluna tabela A que existe na outra coluna da tabela B).



INNER JOIN

A sintaxe do INNER JOIN é a seguinte:

```
SELECT
    *
FROM
    tabelaA
INNER JOIN tabelaB
ON
    tabelaA.ColunaPK = tabelaB.ColunaFK
```

INNER JOIN

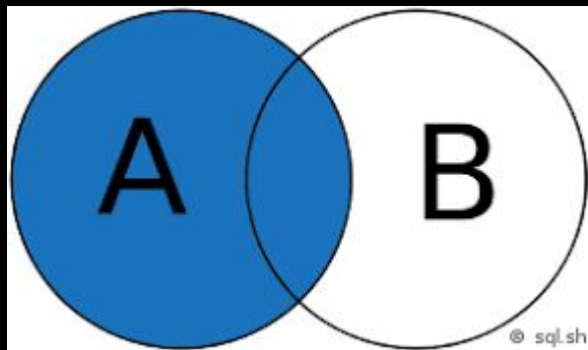
1. Agora com isso replique o comando abaixo:

```
SELECT  
    *  
FROM CLIENTES  
INNER JOIN CARROS ON CARROS.FK_ID_CLIENTE = CLIENTES.ID_CLIENTE
```

LEFT JOIN

O LEFT JOIN irá pegar todos os registros da tabela da esquerda e juntar com os registros da tabela da direita.

Em suma ele vai juntar toda a tabela da esquerda juntamente com o que combinar da direita ou não



colID	colVal	columnID	columnVal
1	one	1	some value
2	two	NULL	NULL
3	three	3	blah blah blah
4	four	NULL	NULL
5	five	5	hello world

LEFT JOIN

O comando para se fazer o LEFT JOIN é:

```
SELECT  
    *  
FROM  
    tabelaA  
LEFT JOIN tabelaB  
ON  
    tabelaA.ColunaPK = tabelaB.ColunaFK
```

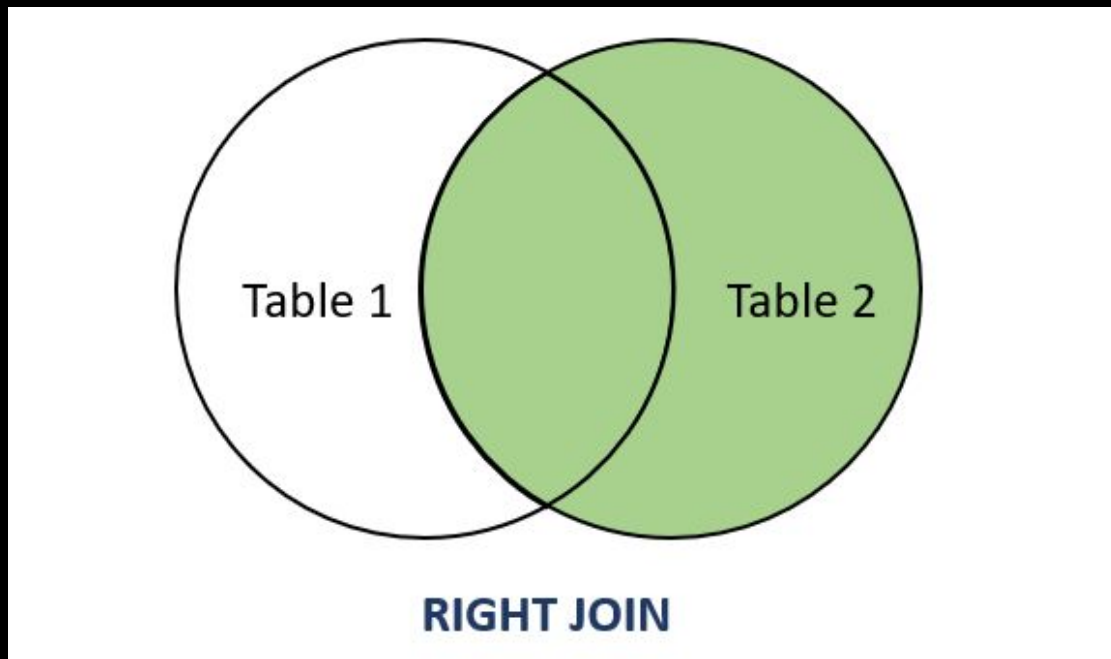
LEFT JOIN

1. Agora com isso replique o comando abaixo:

```
SELECT  
    *  
FROM CLIENTES  
LEFT JOIN CARROS ON CARROS.FK_ID_CLIENTE = CLIENTES.ID_CLIENTE
```


RIGHT JOIN

O RIGHT JOIN permite fazer o mesmo que o LEFT JOIN, porém do lado direito.



RIGHT JOIN

O comando RIGHT JOIN serve é feito da seguinte maneira

```
SELECT  
    *  
FROM  
    tabelaA  
RIGHT JOIN tabelaB  
ON  
    tabelaA.ColunaPK = tabelaB.ColunaFK
```

RIGHT JOIN

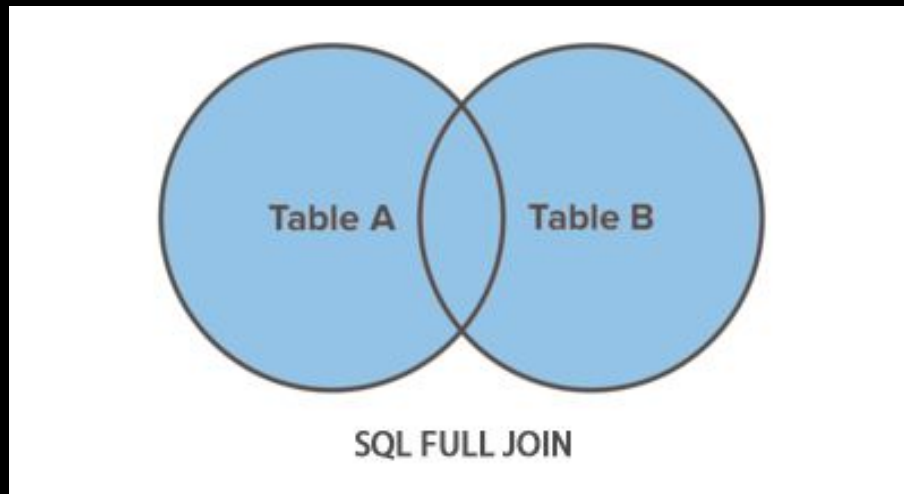
1. Agora com isso replique o comando abaixo:

```
SELECT
    *
FROM CLIENTES
RIGHT JOIN CARROS ON CARROS.FK_ID_CLIENTE = CLIENTES.ID_CLIENTE
```

FULL JOIN

O FULL JOIN retorna todas as linhas das tabelas unidas, correspondidas ou não, ou seja, você pode dizer que a FULL JOIN combina as funções da LEFT JOIN e da RIGHT JOIN.

Em um resumo o FULL JOIN só unirá as tabelas com valores correspondentes ou não.



UNION

O MySQL não tem suporte ao FULL JOIN, então para solucionarmos isso, utilizamos duas queries unidas através da palavra reservada UNION, que nos permite unir duas ou mais queries. Em nosso caso faremos um LEFT JOIN e uniremos com um RIGHT JOIN.



SELECT

*

FROM

tabelaA

LEFT JOIN tabelaB **ON** tabelaA.ColunaPK = tabelaB.ColunaFK

UNION

SELECT

*

FROM

tabelaA

RIGHT JOIN tabelaB **ON** tabelaA.ColunaPK = tabelaB.ColunaFK

RIGHT JOIN

1. Agora com isso replique o comando abaixo:

```
SELECT
    *
FROM CLIENTES
LEFT JOIN CARROS ON CARROS.FK_ID_CLIENTE = CLIENTES.ID_CLIENTE

UNION

SELECT
    *
FROM CLIENTES
RIGHT JOIN CARROS ON CARROS.FK_ID_CLIENTE = CLIENTES.ID_CLIENTE
```

Atividades

1. Crie uma tabela chamada “produtos” que contenha os seguintes campos:

ID_produto	Nome_produto	Preço
INT PK AI	Varchar(50)	FLOAT

1. Agora preencha essa tabela com 10 valores.

Atividades

1. Crie uma tabela chamada “Vendas” que contenha os seguintes campos:

ID_venda	id_produto	Nome_cliente
INT PK AI	INT FK	VARCHAR(50)

1. Agora preencha essa tabela com 10 valores.

Atividades

1. Faça agora um join com a tabela de Vendas e com a tabela cliente e traga todas as vendas e os nomes dos produtos.
1. Com a tabela juntada filtre todas as vendas do produto de id de número 5
1. Com a tabela juntada filtre as vendas dos produtos, onde o preço é maior que 20
1. Com a tabela juntada agrupe todos os produtos por nome e veja os produtos que mais venderam

Bonus: Tabela Dimensão

Uma tabela dimensão é onde ficará contido informações de várias características de nossa tabela pegue por exemplo a tabela de cliente ela terá características como: Nome do cliente, endereço, ID_cliente e etc.



Bonus: Tabela Fato

É uma tabela que irá registrar acontecimentos de uma empresa como uma compra, venda dentre demais atividades,

Normalmente é composta por colunas de ID que servem para buscar as informações complementares em tabela Dimensão. Normalmente é uma tabela que tem milhões de linhas.

